

Yama's Senior Design Documentation

Contents

1	Progress Updates	2
1.1	Week of 2/9/2026	2
1.2	Week of 2/16/2026	2
1.2.1	Define ML Problem	2
1.2.2	Features	4
1.2.3	Pipeline	4
2	Journals	5
2.1	Journal 1	5

1 Progress Updates

1.1 Week of 2/9/2026

I worked on our team's Jira and researched the different components of Epics, Stories, Tasks, Subtasks, etc. I also set up the Design Proposal & Team Contract and the Early Status Report documents. I made a template with the table of contents of everything we needed to fill out for each submission.

1.2 Week of 2/16/2026

I added new tasks for our team's current sprint and assigned them to their respective roles on Jira. I worked on my motivations and ideas section for the design proposal. I started on my research on my machine learning and threat analysis role. I researched how this role worked in cybersecurity and how clustering and anomaly detection worked. From there, I started looking at different resources on how to build pipelines and what information I should be using from each session. I also set up our READ.me and scrum & meeting documentation on GitHub

1.2.1 Define ML Problem

- **Clustering:** an unsupervised machine learning technique that automatically groups similar data points into clusters based on shared characteristics without the use of existing labels.
 - Helps identify hidden patterns or structures in data
 - Groups data by similarity (distance, density, or hierarchy)
 - Algorithm discovers patterns on its own
 - Common Algorithms:
 - * K-Means – partitions data into a fixed number of clusters based on distance to centroids
 - * Hierarchical Clustering – builds a tree-like structure of clusters at multiple levels of granularity
 - * DBSCAN – groups data based on density and can naturally identify noise/outliers
- **Anomaly Detection:** a technique used to identify data points, events, or observations that deviate significantly from the norm
 - Detects rare or suspicious events that do not follow expected patterns
 - Normal behavior (legitimate users, standard network traffic) forms the baseline and deviations from this baseline may indicate attacks
 - Uses: Fraud Detection, Network Security (intrusion detection), Machine Failure Detection,
 - **Types of Anomalies:**
 - * **Point Anomaly (Single Event Anomalies):** a single data point that is significantly different from normal behavior
 - Ex: Session with an unusually high number of commands, single IP making hundreds of requests per second, login attempt with extremely high password entropy
 - Often indicates brute force attempts, bots, or probing
 - Features: request count, command frequency, password entropy, sent/received bytes
 - * **Contextual Anomaly (Condition Based Anomalies):** normal behavior occurring in the wrong context
 - Ex: Login attempts at 3AM when user normally logs in during the day, admin level commands from new geographic location, long sessions during times that usually inactive
 - Attacks often blend in by behaving normally but at strange times or locations
 - Features: time of day/week, geolocation or ASN, historical user behavior
 - * **Collective Anomaly (Pattern Based Anomalies):** a sequence or group of events that is suspicious only when viewed all together

- Ex: slow brute force attack (many login attempts over a certain time), port scanning across multiple endpoints, distributed attacks, botnets generating coordinated low rate traffic
 - Most attacks tend to try and avoid obvious spikes
 - Features: session duration, event sequences, temporal patterns
- * **Behavioral Anomaly (Profile Deviations):** occurs when an entity deviates from its own historical baseline
 - Track: users, ip addresses, devices, API keys
 - Ex: user using commands they never used before, changing request patterns repeatedly, IP switching from read only requests to destructive actions or commands
 - Effective for detecting compromised accounts
 - Compare current vs historical behavior
- * **Structural Anomaly:** occurs when a relationship between features break down
 - Ex: normal request rate but abnormally high error rate, normal session duration, but unusual command diversity, high authentication success with low credential entropy
- **Approach:** We will start with anomaly detection and add clustering as a bonus insight if time allows. Clustering can also be used as a technique for anomaly detection by helping identify outliers. Anomaly detection directly aligns with our goal of identifying, categorizing, and predicting malicious behavior in security data.
- **Bot-like Behavior:** Activity patterns that look automated rather than human.
 - Machine learning helps detect behavior that statistically deviates from normal human behavior
 - **Timing Patterns**
 - * Bot-like: near constant times for events, very precise, active 24/7
 - * Human-like: delays, pauses, gaps in patterns
 - * Features: mean/variance of time between requests, entropy of inter arrival times, burst score
 - **Action Repetition**
 - * Bot-like: optimize, same requests many times, same commands or endpoints, fixed navigation path
 - * Human-like: variations, back tracking, exploring, mistakes
 - * Features: N-gram repetition rate, unique action ratio, command diversity
 - **Volume and Speed**
 - * Bot-like: hundreds or thousands of actions per minute, parallel sessions from same IP or account
 - * Human-like: short bursts, hard limit on actions per limit
 - * Features: Requests per minute, concurrent sessions, session duration
 - **Input Characteristic**
 - * Bot-like: no typos, perfectly structured inputs, uniform string lengths, low entropy (dictionary based) or extremely high entropy (auto generated) passwords
 - * Human-like: typos, inconsistent formatting, mixed entropy
 - * Features: password entropy, input length variance, character distribution
 - **Goal Driven Behavior:**
 - * Bot-like: exists to do one thing, scraping, brute force, credential stuffing, enumeration
 - * Human-like: multiple unrelated goals, exploring before acting
 - * Features: Ratio of failed/successful actions, target diversity, sequential error patterns
- **Common Bot Types**

- Scraper: high read, no writes
- Credential Stuffer: Many logins, many accounts
- Scanner: touches many endpoints at one
- Spam Bots: repetitive form submissions
- Click Fraud: uniform click timing

1.2.2 Features

List of features (not comprehensive)

- **Session Duration:** total elapsed time between first and last event in a session
 - Bots usually have every short sessions with many commands or very long sessions with no inactivity (running 24/7)
 - last time stamp - first time stamp
- **Command Count:** total number of commands, requests, and events within a session
 - Unique command ratio, repetition rate, commands per min
- **Password Entropy:** measure of randomness in attempted passwords, typically using Shannon entropy
 - Entropy = $L * \log_2(R)$ where L is password length and R is pool of possible characters (lowercase, uppercase, alphanumeric, etc)

1.2.3 Pipeline

1. Data Ingestion & Normalization

- Input: Authentication logs, command/API request logs, network/session data
- Parse timestamps, normalize command names, hash sensitive fields (passwords, tokens)
- Output: Clean, structured events

2. Sessionization

- Convert events into behavior units
- Group events by sessionid, ip, or user
- Apply timeouts since ML relies on sessions

3. Feature Engineering

- Core Features: session duration, command count, password entropy
- Timing Features: mean/variance of inter-command time, burstiness index, idle time ratio
- Structural Features: unique command ratio, repetition rate (n-gram similarity), endpoint diversity
- Authentication Features: failed and successful login ratio, account switch count, credential reuse count
- Network / Context Features: IP reuse across sessions, geolocation variance, user agent consistency

4. Feature Scaling

- Standardize, remove noise and inconsistencies
- Handle any missing values, outliers, and duplicates

5. Detection Stage

- Identify bot like behaviors without labels

- Unsupervised Models: isolation forest, autoencoder, one class SVM
- Outputs anomaly score per session and bot likeness rankings

6. Classification Stage

- Occurs after detection
- Inputs anomalous sessions and feature set
- Classify based on patterns and features and create labels
- Supervised or semi-supervised
- Random forest, XGBoost, multiclass neural net

7. Evaluation & Validation

- Feature distribution plots to understand normal vs anomalous behavior
- Anomaly detection metrics: precision@k, false positive rate, and anomaly score distributions
- For optional attack classification, supervised metrics including confusion matrix, per class F1 score, and macro averaged recall

2 Journals

2.1 Journal 1

This week, I organized our team's workflow in Jira by researching purpose and use of epics, stories, tasks, and subtasks by using online documentation and the PowerPoints in canvas. After this weeks meeting, I added new sprint tasks and assigned them on Jira based on each role's responsibilities. I set up our group's Design Proposal & Team Contract and the Early Status Report documentation by creating a template with the table of contents based on the assignment requirements on canvas so everyone can start working on it. I added a READ.me and a scrum/meeting documentation on our GitHub so the team has access to view our minutes at anytime.

For my machine learning and threat analysis role, I researched how they are applied in cybersecurity in clustering and anomaly detection. I concluded that building an effective pipeline requires things like session duration, command and prompt counts, and behavioral patterns. These features will be important in our project to help identify attack patterns, detect abnormal sessions, and possibly predict attacks down the line after our model is trained. I used various resources like academic papers, online documentation, and videos from IBM Technology to better my understanding during my research process. I have no current blockers and next week I plan to work on prototyping by writing up a structured framework for the pipeline including preprocessing steps in taking in the data, computing features, and plotting the distributions. I will use synthetic or test inputs to ensure everything works properly so we can integrate the real data later on.